- When the `Map` extension matches a request path segment, it removes the segment from the request's `HttpContext.Path` and moves it to the `PathBase` property. This ensures that routing in branches works correctly.
- You can use the `Use` extension method to create generalized middleware components that can generate a response, modify the request, or pass the request on to subsequent middleware in the pipeline. This is useful for cross-cutting concerns, like adding a header to all responses.
- You can encapsulate middleware in a reusable class. The class should take a `RequestDelegate` object in the constructor and should have a public `Invoke()` method that takes an `HttpContext` and returns a `Task`. To call the next middleware component in the pipeline, invoke the `RequestDelegate` with the provided `HttpContext`.
- To create endpoints that generate a response, build a miniature pipeline containing the response-generating middleware, and call `endpoints.Map(route, pipeline)`. Endpoint routing will be used to map incoming requests to your endpoint.
- You can configure `IOptions<T>` objects using a fluent builder interface. Call `AddOptions<T>()` to create an `OptionsBuilder<T>` instance and then chain configuration calls. `OptionsBuilder<T>` allows easy access to dependencies for configuration, as well as features such as validation.